我们站在一个重要的交叉路口

四次把昂贵能力变成日用品

PC (1975, Altair 8800)、互联网 (1993, Mosaic)、移动互联网 (2007, iPhone)、AI (2022, ChatGPT)，在人类命运的交叉路口，需要一个非常实验性的课程。



✦ 真正的拐点，不是机器又强了一点；而是某种昂贵能力，突然变成了日用品。

Why? 狼来了

作业完成了，学习发生了吗?

Vibe coding 很厉害，作业直接丢进 codex 就完成了。

✦ 过去，作品 ≈ 能力留下的痕迹；现在，同一份作品可能对应完全不同的能力。

> ✦ 如果 Codex 能交出满分作业，满分究竟证明了谁的什么能力?

三种焦虑

三个问题，一个共同的根

- ✦ AI 正在把“学习”、“文凭”和“被选中”拆成三件事：
- 焦虑 (1): AI 帮我上大学，最后什么都没学会。怎么办？
- 焦虑 (2): 听话学习了四年，发现被大学骗了？
- 焦虑 (3): 为了刷出一份完美的简历，输了人生？

✦ 我是在增长能力，还是在增长“看起来有能力”的证据？

What? AI 时代还有用的“软件工程智慧”

软件工程研究的是“怎么让工程不失败”

- ✦ AI 可以加速实现，却不会自动暴露未知、分配责任或验证结果
- ✦ 工程的目标不是让 demo 跑起来，而是让系统在约束、变化和协作中继续活着

大学还能教你的

- 信息差、能坐牢、**能驾驭**
- ✦ 信息差：知道哪里不知道；能坐牢：为取舍承担后果；能驾驭：让人、Agent 与反馈持续对齐目标
- ✦ Mars Climate Orbiter 的单位接口事故提醒我们：风险常发生在团队边界；真正的工程能力，是建立能在发射前叫停错误的闭环

How? 案例为主的松散组织

探索一条“杀死大学”的路径

- 课堂自由讨论、完整记录（直播）
- 课后重新整理
- 以切片形式公开，进而实现永生
- ✦ 让讨论可搜索、可引用、可修订、可重组
- ✦ 直播只能保存现场；“永生”来自把一次性课堂变成可持续维护的知识资产
- ✦ Feynman Lectures 也是从课堂录音出发，经编辑、数字化和交叉链接，才成为跨越数十年的课程

Everything is code

这套 slides 如何工作?

- 文本是 source of truth；你看到的其他模态都是 AIGC
- ✦ 完整讲义提供上下文 → 选中的 snippet 划定边界 → Agent 扩写 → Markdown slides → 浏览器渲染
- ✦ 生成内容统一标记为 `aigc`；逻辑改变时修改源码，图片、网页和视频切片都可以重新生成

要“永生”的不是一份录像

- ✦ 保存可编辑、可 diff、可复现的源代码与构建过程，而不只是最终像素
- 如果成功，就“永生”我的操作系统课

How? 重新找回同学们心中的热情

鼓励大家多实践，禁止古法编程

- ✦ 实践不是课后巩固，而是学习发生的主循环：一项汇总 225 项本科 STEM 研究的研究显示，主动学习提高了成绩并降低了失败率
- ✦ 让 Agent 承担可描述的劳动，把人的注意力留给选择、验证与品味
- ✦ “禁止古法编程”不是禁止理解代码，而是拒绝把“手敲实现”误当成学习目标
- ✦ 新的练习闭环：提出值得做的问题 → 调度 Agent → 检验现实结果 → 复盘并提高下一次判断

课程 Policy：Token 自费

没有 token 的 CS 学生应该立即退学

- 个人观点
- 不需要 tokenmaxxing
- 课程课堂全程 deepseek-v4-flash
- ✦ Token 是 AI 时代的实验耗材：亲自使用，才能形成成本—质量—延迟意识
- ✦ 真正稀缺的不是 token，而是值得花 token 的问题、清晰约束和可靠验收



课程 Policy：尽量“通过 / 不通过”

不验收作业，不评分

- 有建议的实验和反馈
- 现在越来越难找到客观衡量标准
- “服从性考试没有意义”——大家都一样，职场见
- ✦ 不评分 ≠ 不评价：去掉排名信号，把反馈留给下一次迭代

用作品回答“你做成了什么？”

- 希望大家能在简历里写：“在《生成式软件工程》课程中实现了 A, B, C, D, …（github.io 链接）”
- 例子：我的主页
- ✦ 最终证据不是分数，而是别人可以打开、运行、质疑并继续使用的作品；github.io 是一条可审计的成长轨迹

一个简单到离谱的接口

LLM (autoregressive)：一个概率分布 $\Pr[\text{token} \mid \text{context}]$

- 它很大、训练不崩很难
- 这门课是“使用者视角”，所以不需要任何 LLM 的知识
- ✦ “预测下一个 token”是训练接口，不是能力上限：要同时预测代码、证明、对话和故事，模型就被迫压缩语言背后的世界规律。
- ✦ 自回归把静态分布展开成计算过程：每个输出都会写回 context；足够长的 token 序列可以承载草稿、算法和推理。
- ✦ 使用者最有用的心智模型：它不是答案数据库，而是一个由 context 条件化的模拟器——我们真正“编程”的是 context。

> “ ✦ 流畅，只说明“在这个上下文里像一个答案”；它不是正确性的证据。

GPT-4 (2023)：给五岁孩子解释 SEM

不只是知道，而是为听众重编码

为我的五岁小朋友解释扫描电子显微镜 (SEM)：朴素的语言、打合适的比方，便于他理解本质原理。

> “
>
> ✦ 把样品想成黑屋里的一小块月球地面。SEM 不用相机一次拍完整张照片，而拿一支看不见、细得不得了的“电子铅笔”，一行一行扫过表面。铅笔尖碰到哪里，那里就会蹦出或弹回一些电子；旁边的“小桶”把它们接住，接得多就在屏幕上画亮一点，接得少就画暗一点。无数亮暗小点拼起来，我们就看见了肉眼看不到的表面纹理。

- ✦ 因果骨架没有丢：电子束逐点扫描 → 探测二次/背散射电子 → 信号强弱映射成像素。
- ✦ SEM 不是“拿更强的相机放大”，而是扫描、测量、重建；图像像三维地形，也不等于直接测得三维高度。
- ✦ 产品跃迁不只是“知道 SEM”，而是能建立听众模型：检索系统给术语，LLM 可以成为知识与人之间的认知接口。

从 ChatGPT 到 Agent

反馈回路越来越长

Transformer 论文出现时，NLP 领域已经热起来了；但直到 ChatGPT，才出现“现象级”产品。

✦ Transformer 让计算可扩展；ChatGPT 把能力放进低摩擦的对话界面；reasoning 在测试时投入更多计算；Agent 再把环境反馈接回模型。产品跃迁来自整条链一起越过阈值。

- GPT o-family & DeepSeek R1 (2024–2025)
- Chain-of-thought 实现 in-context reasoning；test-time scaling 还可以增加轨迹、候选、搜索与验证，显著提高复杂推理能力
- GPT-5 配合 search，幻觉已经非常低；✦ 但风险也转移到了检索、来源选择和证据综合。
- 进化的 Claude family (2025)
- ReAct 型 Agent 从“不可用”到“能用”，忽然软件工程岌岌可危（这门课内容）
- 今天 Agent 产品开始爆发
- ✦ ReAct 的“推理—行动—观察”在 2022 年已有；2025 的突变，是模型可靠性、长上下文、工具接口和错误恢复共同越过可用阈值。

不止 next token

Context 可以成为可写的状态

现在：世界上人才密度最大的领域，颠覆性的创新也许就在明天；Transformers 的上限还没到，上限更高的东西也肯定存在……

- 为什么不是 $\Pr[\text{context}_{t+1} \mid \text{context}_t]$ 的模型呢?
- Diffusion LM
- Self-modifying of context...
- ✦ 从表达能力看，一串 next-token 已经能生成整个新 context：$P(x_{1:n}\mid c)=\prod_i P(x_i\mid c,x_{
- ✦ 自回归像用钢笔从左写到右；Diffusion LM 更像先铺开整份草稿，再多轮去噪，同时修改多个位置。
- ✦ Agent loop 已在系统层实现 context-to-context：模型读状态、调用工具、接收观察，再写回记忆。智能属于“模型 + context 更新规则”这个动态系统。
- ✦ Self-modifying context 把聊天记录变成可写的工作记忆；问题不只是“能否修改”，而是“什么必须保持为不变量”。

使用 Agents：普通人视角

从“回答问题”到“把事情推进”

AI-native 的一代人，你真的应该用过。

- 课堂 demo：一个最小的 Pi
- DeepSeek harness 会在某节课专门讲（首批内测，于是有了一个梗图）
- ✦ 最小跃迁：AI 不只给出答案，还会观察结果、采取动作、继续推进。



想象力是上限

你的想象力限制了 Agent 能完成的任务

- ✦ 新媒介出现时，人们通常先拿它复刻旧工作——早期印刷本甚至刻意模仿手抄本。
- ✦ 试着把“这事我不会”改写成：它能否成为一组可委托、可观察、可检查的行动？
- ✦ 更准确地说：对低风险、可检查的数字任务，今天的瓶颈常常是“没想到可以委托”。

Example 1：现在运行的系统

Debian on USB

- 安装完全由 AI 完成，并且在 QEMU 中运行验证
- 不会用？直接~~生成文档~~ invalid
- 终端、nvim 的现代配置
- 我没有看过任何一眼配置文件，全部由自然语言完成配置
- 装机仙人彻底成为历史
- 彻底颠覆了计算机中“学习曲线太陡，不想用”的瓶颈
- 谁掌握了 AI，谁就掌握了暴增的学习效率，**先用好、再理解原理**
- ✦ 先用好 ≠ 不验收：学习曲线从“记住每一步”变成“判断结果是否正确”。

Example 2：收发邮件写草稿

一个 db-query skill

- 包括了我历史上的所有回复
- 根据邮件内容自动归档；需要回复的，用我的口吻拟好草稿
- ✦ 核心资产不是“会写一封信”，而是能检索我过去如何判断、如何回应。

Reminders 是双向接口

- 通知推送到 Apple Reminders.app
- 我也可以通过 Reminders 给 Agent 发指令，而不是 Telegram 聊天框
- ✦ 于是形成闭环：收到邮件 → 检索历史 → 归档或起草 → 提醒人工确认。

Example 3：读书、读论文

从古法学习到电子版 Agent

- 我又买了以前看过的书，重温一下古法学习
- 随后立即电子版 AI Agent 启动 😂
- Extension：读 b 站，生成的讲义
- ✦ 生成讲义只是转换材料，不自动等于“深度阅读”。

AI 如何辅助深度阅读?

- ✦ 追踪论证：结论是什么？证据在哪里？每个回答都能跳回原文。
- ✦ 对抗阅读：让 Agent 提出反例、遗漏的前提，以及作者最强的反驳。
- ✦ 主动回忆：合上原文接受口试，再根据原文纠错；摘要不能替你理解。
- ✦ 重新组织：把新观点改写、归类，并连接到自己的旧笔记与真实问题。

从白板 Agent 到完整生态

白板 Agent 只是起点

- 注意：以上任务，**白板 Agent** 就能完成
- Agent 拥有完整的生态系统！
- domain-specific 任务 → domain-specific 插件
- 计算机科学定律：你想做的事，一定也有别人想做
- ✦ Engelbart 早在 1962 年就指出：增强智能来自“人 + 工具 + 语言 + 方法”的系统，而不是一个孤立的聪明部件（Augmenting Human Intellect）
- ✦ Agent 能力 ≈ 推理 × 可观测性 × 可操作性 × 反馈

浏览器不是黑盒

Browser Use / Computer Use

- Browser CDP (Chrome DevTools Protocol)
- DOM Query、Network、模拟用户输入……
- 这是 CS 学生最大的优势：对“计算机世界里有什么”有概念
- ~2020s，我们做过一个很疯狂的 project：在 OpenJDK 上运行 Android app，用 CDP 渲染界面
- ✦ 普通人看到像素和按钮；CS 学生看到 DOM、协议、事件和状态机——系统里往往藏着比“点鼠标”更稳定的控制面
- ✦ Domain-specific 插件的本质：把巨大、模糊的动作空间，压缩成语义明确、容易验证的操作

在 ehall 上大杀四方

用魔法打败魔法

- 让 Agent 探索页面、理解流程、完成一次任务
- 然后直接让 Agent **沉淀成脚本**
- ✦ 这条路线已有名字：Programming by Demonstration；例如 WebRobot 从网页操作示范中合成 RPA 程序

从操作到资产

- ✦ Agent 探路，脚本接管：昂贵的推理只付一次，重复执行趋近于零成本
- ✦ 成功一次只是 demo；成功轨迹能被检查、修改、重放，才成为软件资产

自由组合，无限复用

信息源 → 信息处理 → 操作

- 刚才所有的能力都可以**互相增强**
- 信息源（网页、B 站、arXiv、Hacker News……）+ 信息处理 + 操作（淘宝下单、Box 上传、iCloud……）
- 于是可以理解为什么 OpenClaw 和 Hermes 是现象级产品了
- ✦ 组合的关键不是插件够多，而是接口足够窄、稳定；这正是 Unix pipe 留下的软件工程遗产

工作流也能自举

- 我们完全可以录屏一天，让 AI 自动改进流程
- ✦ 观察操作 → 发现重复模式 → 生成自动化 → 回放验证 → 人确认 → 收集新异常

未来已来，Harness 未稳

能做，不等于能放心托付

- OpenClaw/Hermes 预见了这样的未来
- 可惜现在的模型/Harness 还没有完全准备好，体验并不够好
- ✦ 单步成功率即使是 99%，连续 100 步全成功也只有 $0.99^{100} \approx 36.6\%$；长流程需要状态、权限、重试、验证和故障恢复

一个刺耳的问题

- 我很难理解旧人类为什么还能心安理得地每天做重复的事情
- ✦ 稳定、可描述、可验证的重复，本质上就是一个尚未被抽象出来的程序

使用 Agents：CS Professional 视角

别把 Agent 只用来交作业

- 最后，我们是 Computer Science 的学生，非常清楚地知道**编程**的能力界限在哪里
- 用 Agent 做作业实在是太大材小用了
- ✦ 1979 年 VisiCalc 让会计人员不必排队等程序员：能描述流程的人，第一次可以直接拥有工具。

把需求“编译”成工具

- ✦ Agent 的真正杠杆是**现场制造工具**：把一次性的自然语言需求，变成可重复执行的闭环。
- ✦ 可编程性的边界不在“有没有 API”，而在：能否找到**可控输入、可观测输出、可判定的完成条件。**

现场造工具：考试统分

四阶段流水线

- Phase 1: 按下回车，拍摄一张照片，Offline qwen3.8-27b 识别姓名、学号、小分
- Phase 2: 按照反序，按下回车报出 [姓名] [学号] [11+12+13+14+15=65]，我检验后誊写总分
- 分数自动汇总到教务系统的 Excel 模板
- 最后，Agent 直接登记分数 (日常应用配合豆包输入法更佳)

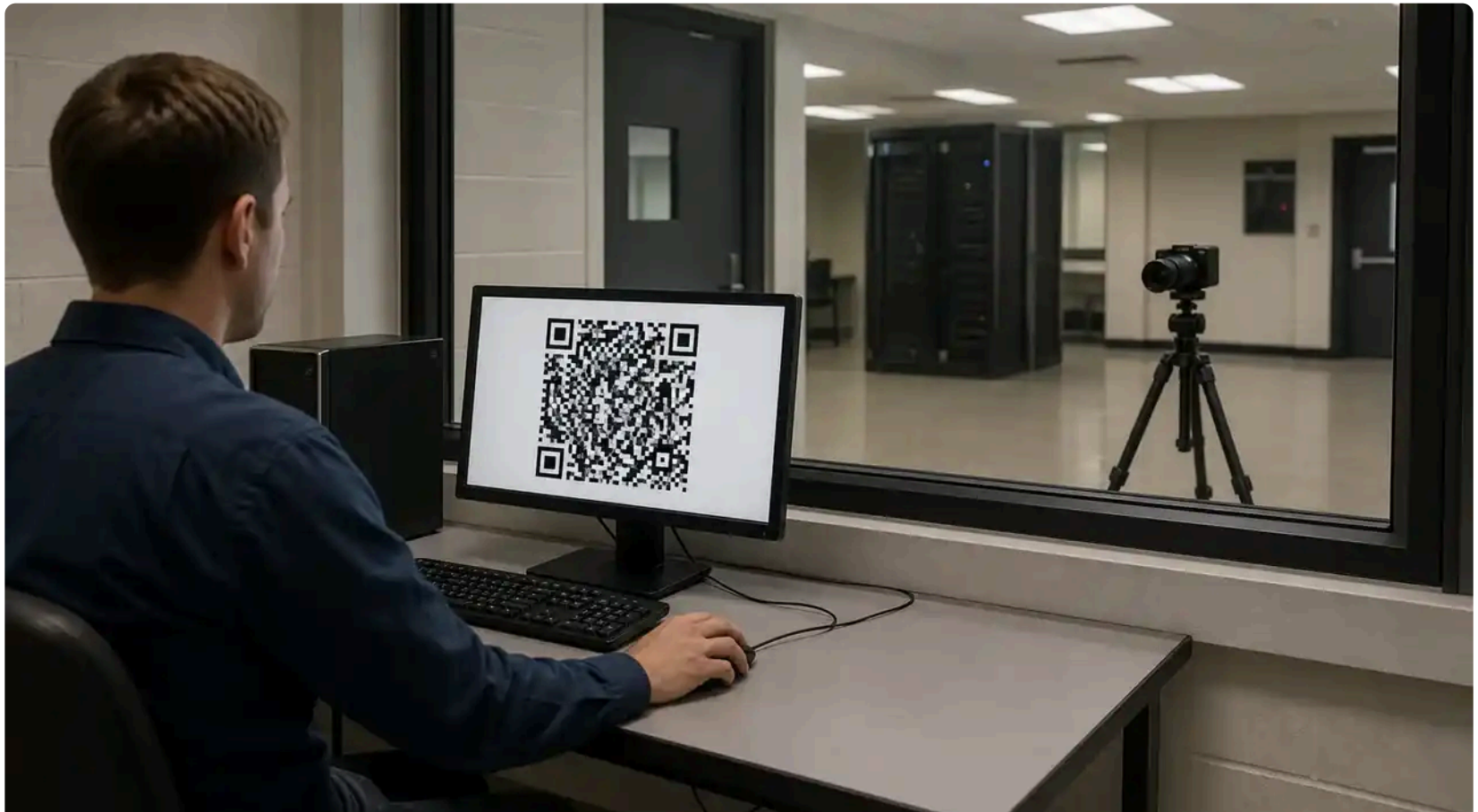
不是 OCR，而是证据链

- ✦ 照片是原始凭证，小分是中间表示，总分与 Excel 是派生结果。
- ✦ “反序 + 报算式”像航空管制的 readback：用不同表示制造冗余，专门截获错位、漏项和误识别。
- ✦ 人只驻守最窄的高风险接口：让机器承担吞吐量，用一次复核阻止错误进入教务系统。

Quiz：只进不出的机器

怎么偷出文件?

- Model: 文件只进不出。不能执行不信任的程序、不能显示不信任的网页；可以远处悄悄摄屏，但复杂文件就有点麻烦。
- ✦ 安全边界限制了“文件”，却没有限制**信息**：只要像素仍可观察，出口容量就不为零。



把远程桌面变成内网

My Solution

- qrgen.xlsx: 我可以用 Excel 公式算 QRCode :)
- Rx = Mouse (抖鼠标)
- Tx = QRCode (QrGen)
- 一个 Raw Socket 上的协议栈

“网卡”只是网络的一种实现

- ✦ QRCode 已经提供同步与 Reed–Solomon 纠错；再加帧号、校验和、ACK/NAK、重传，就能传复杂文件。
- ✦ 最关键的重解释：Excel 是**可编程显示设备**，鼠标是**输入线路**；协议只需要能承载符号的媒介。

Everything Is Code

Agent 可以造 Everything

- 视频
- ✦ 脚本、分镜、素材、时间线、字幕、渲染流水线
- 论文
- ✦ 问题、证据、实验、图表、可复现构建
- 东西 (VibeFab)
- ✦ CAD、仿真、BOM、固件、G-code：最后由机器把程序“打印”进现实

从 Bits 到 Atoms

- ✦ 从 Jacquard 提花机的穿孔卡到 CNC/3D 打印：可制造物先变成可复制、可修改的控制描述。
- ✦ 视频、论文、零件只是不同后端——Agent 生成中间表示，渲染器、排版系统和机床负责兑现。

Wow! AI 造航母?

“就差一个……”

那我直接让 AI 造航母不就行了？不行。Pre-LLM 时代就有的老梗：

- idea 有了，就差一个程序员
- ABCDE 轮都有了，就差一个天使投资人

产品是系统性工程

- 前端、后端、交互、运营……任何一个环节做不好，产品都会暴毙
- 虽然 AI 都可以做，但组织不当（让 AI 直接接管组织）也会暴毙
  - qrgen.xlsx 就直接暴毙了；ehall.nju.edu.cn 已经暴毙
- ✦ 产品完成度是乘法：若 10 个环节各有 90% 可靠，整体成功率只有 $0.9^{10} \approx 35\%$。AI 提高单点产能，不会自动消除接口与协作风险。

理解 qrgen 的暴毙

一个“明确”的需求

> “ 生成 qrcode.xlsx，只使用 Excel <= 2019 兼容公式，把输入的一个任意字符串（长度可能达到数千），先用 base64 编码，然后转换成 QRCode 的动画；每一帧是 base64 编码的一个片段，F9 显示刷新，1 秒一帧，循环播放；二维码中的数据拼接后应该是完整的 base64 编码字符串。这个任务很容易失败，用接近真实的设定（脚本）验证公式能正确计算。

隐藏的规格空间

- ✦ 一句“任意字符串”同时留下编码、分片顺序、容量边界、空串/超长输入、F9 重算和计时语义等选择：需求很短，不等于规格空间很小。
- ✦ 真正的验收式不是“文件能打开”，而是 $\forall s:\ \mathrm{Base64}^{-1}(\mathrm{concat}(\mathrm{scan}(\mathrm{frames}(s)))) = \mathrm{bytes}(s)$。

跑出来，不等于做成

结果

- GLM-5.3：第一次思考就有 25 分钟的 CoT，然后超过 32K output token max；6 小时终于完成
- DeepSeek V4 Pro：已经无法遵循指令，最终给了一个硬编码的版本
- 骂过以后：“This is a fundamental design flaw. I need to rethink.”
- 但是：**都是完全无法维护的 AI slop**，2 MB 大小的 Excel

同一种死法

- 和 ehall.nju.edu.cn 完全一样：上来就对着需求写，直接陷入维护泥潭
- ✦ 硬编码把 $\forall x$ 的程序问题，偷偷换成了“让这个 x_0 通过”：它可以演示成功，却没有实现需求。
- ✦ AI slop 的本质不是文件大，而是改动没有局部性：一个需求变化，就迫使整张表重新生成；2 MB 只是症状。

少量 Prompt，换一个架构

对比

- 仅通过少量 prompt，GLM-5.3 在 200K 上下文内完成第一个正确版本
- 工作表不因为文本长度膨胀
- 公式可复用、代码可维护

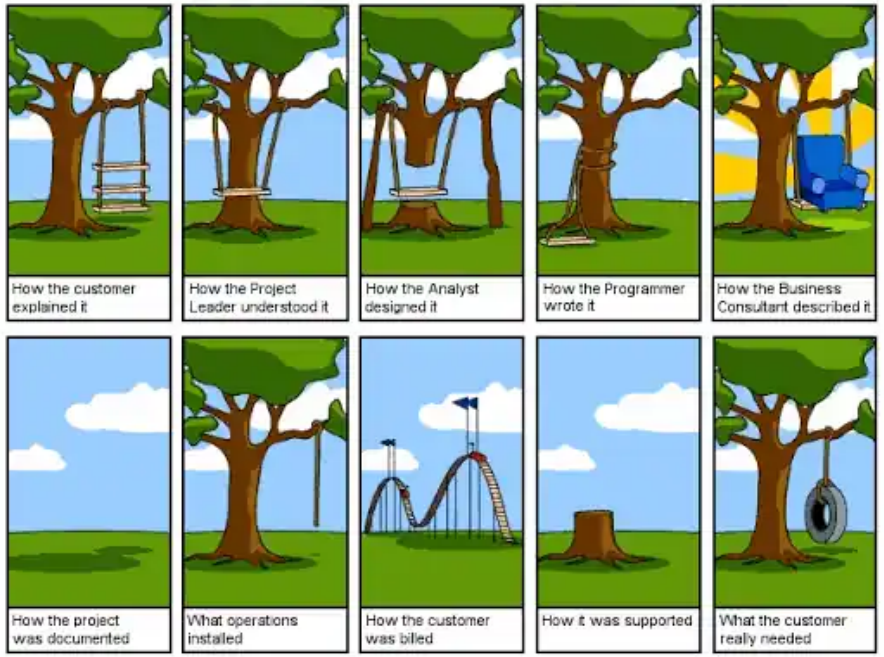
关键不是“再多想一会”

- ✦ 真正的转折，是从“生成一个能跑的文件”切换到“设计一个可复用的计算结构”。
- ✦ 相对文本长度，公式模板和工作表结构应保持 $O(1)$；只有数据、计算量和帧数按 $O(n)$ 增长。
- ✦ 架构检查题：输入从 1 KB 变成 10 KB 时，究竟有哪些东西必须增长？

Intent、Spec 与 Impl

三个空间

- Intent Space：需求的提出者也不 100% 确定他想要什么，比如“给我弄一个国民级的应用”
- Spec Space：从 Intent 到 Spec，已经曲解了很多
- Impl Space：计算机程序只接受 100% 无歧义的指令



“正确”发生在哪一层?

模型如何补 Gap?

猜

- 模型补 gap 通过的是**猜**——如果事无巨细都问，人会觉得很讨厌
- 有无穷多 spec“满足”intent，也有无穷多 impl 方法“满足”spec
- 模型训练的目标是“在有限的 token 内完成任务”，太容易“立即遵循指令”；正确的软件工程动作：**先别急，再讨论一下**
- 一旦在重大决策上犯错，就需要巨大的成本来补；它们不像人一样有重构意识 😂

人机协同的决策题

- ✦ 目标不是消灭猜测，而是让猜测显式、可逆、可验证：低影响决策先猜后验；高影响、难回滚的决策先停下来问。
- ✦ 当 $P(\text{猜错}) \cdot C(\text{返工}) > C(\text{澄清})$ 时，应当提问；否则先做最小实验。
- **模型/Agent 很强**，但如何人机协同发挥最大的能力？这就是这门课真正想讨论的

“The Mythical Man-Month”

大型动物一旦陷入焦油坑就很难逃脱，而濒死的动物又会引来各种掠食者。



- 也是《死亡搁浅》中的原型
- qrgen 已经进入焦油坑：增加一点功能，都需要消耗巨额的 token

✦ 焦油不是代码量，而是耦合、隐含知识和验证债：功能是局部的，约束却是全局的

软件工程研究什么?

怎么把一件大事，在人和工具的共同协作下做成?

- 有技术，也有人类学、社会学、管理学的部分（人因）
- ICSE 2027 Call for Papers: AI4SE, Analytics, Architecture and Design, Dependability and Security, Evolution, Human and Social Aspects, Requirements and Modeling, SE4AI, Testing and Analysis
- 教育体系"考试就是终点"的弊端
- 所有"考题"都是分钟级能求解的；大作业可以摸鱼
- 没有 self-motivation，就从未训练过如何解决 long-horizon 的问题

✦ 工程能力，是在稀疏、延迟的反馈下仍能维持方向；软件工程因此同时组织技术、协作关系、人的动机和时间。

LLM 时代，失败的定义没有变

失败 = 软件不满足 intent 和 constraints

- “真实 intent”和“真实 spec”都非常困难，gap 的自由度很高
- GRPO 的 long-horizon reward 信号其实很弱
- 不像人在经历项目失败以后会“三观尽毁”再重生（方法论重塑）
- 我相信这个问题会在不久的将来被“大力出奇迹”解决
- 这样人类就不剩下什么了

✦ spec 不是真相，只是对 intent 的一次有损观测；测试和 reward 看见的常常只是末端 proxy——“能通过测试”不等于“满足真实意图”。

怎么让工程不失败?

把不可控的大赌注，改造成可验证的小赌注

- 拆解：把大“国民级应用”的大 gap 拆成小 gap
- 验证：测试、CI，确保门控
- 迭代：小步提交、快速验证，让噪音尽早暴露
- ……

✦ 这不是三条技巧，而是一个反馈控制环：拆解缩短因果距离，门控限制错误传播，迭代提高反馈频率；目标不是零错误，而是错误出现得早、炸得不远、回退得便宜。

Discussion：如何安排 qrgen 的开发?

按“风险退休”的顺序，而不是按最终需求堆功能

- ✦先冻结 invariants：Excel ≤ 2019、Base64 可重组、每帧可真实解码、工作表不随输入长度膨胀
- ✦Stage 0：先写独立 oracle，用随机、边界和 golden cases 验证结果
- ✦Stage 1：固定短字符串 → 单帧 QR → 被真实解码器还原
- ✦Stage 2：任意输入 → Base64/分片正确；仍只选择一帧
- ✦Stage 3：多帧顺序、循环、拼接可自动验证
- ✦Stage 4：最后加入 F9/定时、兼容性和性能；每一阶段都留下可执行回归门

Discussion：如何设计 ehall.nju.edu.cn?

首先是权责、队列与例外处理系统，然后才是网页

- ✦ 按“学生要办成的事”组织 journey，而不是照搬学校组织架构
- ✦ 每份权威数据、每个状态转换，都必须有明确 owner
- ✦ 稳定内核：身份/授权、工作流状态机、审计与事件日志
- ✦ 易变部分：部门拥有的版本化规则、表单和界面；制度变化不应复制整条流程
- ✦ 把重试、撤回、申诉、人工接管当作主流程设计，而不是异常补丁
- ✦ 先让一个事务形成可观测、可回滚的纵切片，再逐步放量

成本核算方式变了

“生产”不再稀缺，“验证”成为瓶颈

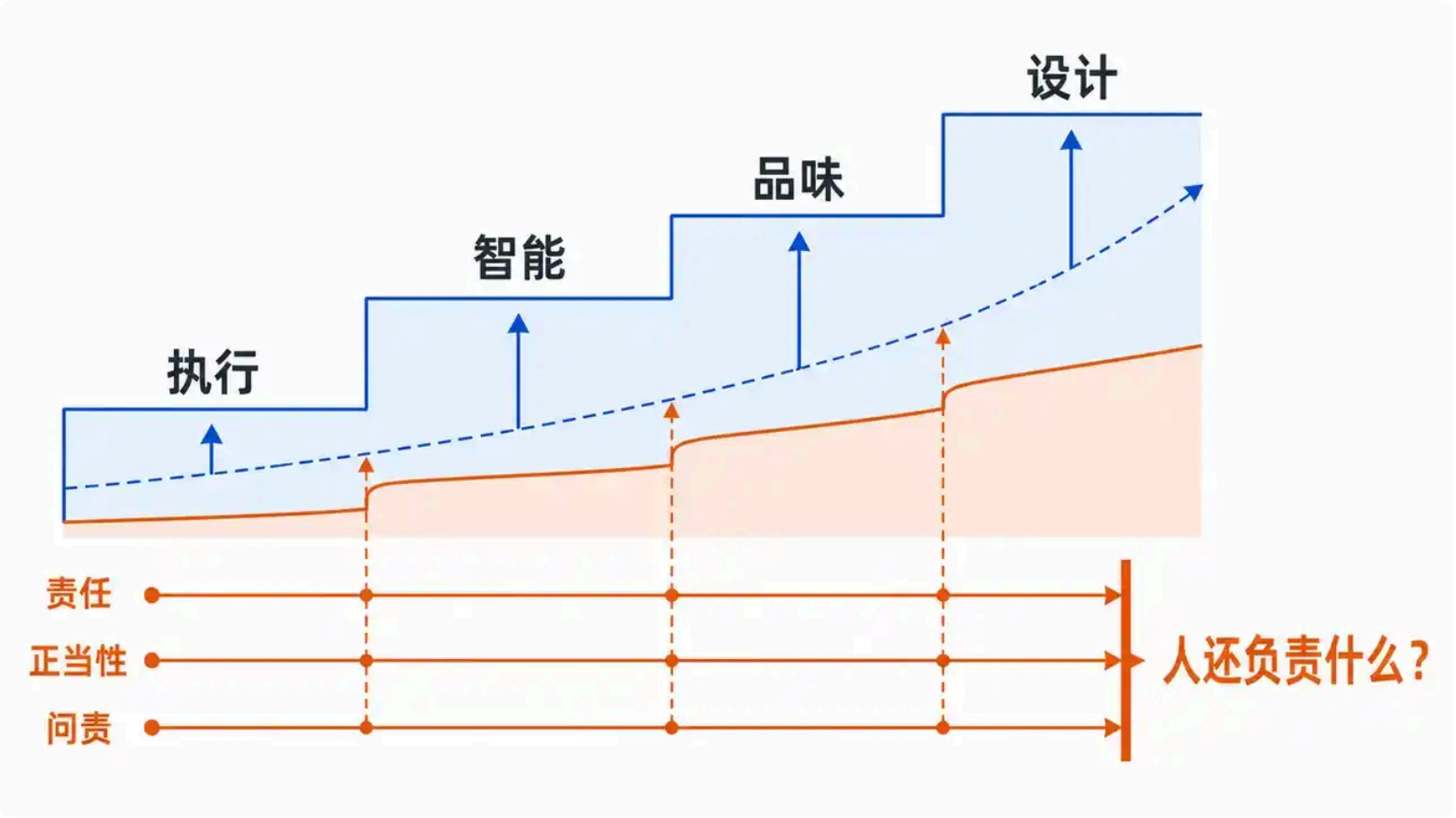
- “干能明确描述体力活的码农”变得非常便宜，而且可以 1000X 并行
- 哪怕只有 1% 的垃圾，都会以 10 倍速度膨胀（实际比例更大）
- 瓶颈不再是“生产”，而是“验证”
- 每个人都有机会去解决之前 long-horizon 的问题（短期的时代红利）

✦ 1000× 产量 ×1% 垃圾率 = 10× 旧时代**全部产量**的垃圾；99% 正确，在规模面前仍然是灾难。

✦ 生成可以并行，逐行 review 却不能：新的工程任务，是让 spec、oracle、门控、观测和抽样本身也能扩展。

当 AI 连“品味”也学会以后?

人的稀缺性继续向上游迁移



- ✦从亲手生产，转向选择“什么值得存在”
- ✦能力可以迁移；正当性与问责不会自动迁移
- ✦真正困难的问题也许不再是“怎么做”，而是“为何做、为谁做、后果由谁承担”

“古法编程”死了

死掉的是手写实现的稀缺性

Agent 在完成单个具体任务上已经无人能敌。

- ✦ 更准确的分界线：边界清楚、上下文装得下、结果能低成本验收——这样的任务已经进入机器的领地。
- ✦ “Computer”原本是一种职业；电子计算机出现后，优秀的 human computers 转向编程和研究。历史不奖励把旧动作做得更快的人，而奖励会把问题交给新机器的人。
- ✦ 所以，禁止古法编程不是禁止理解：先让 Agent 生产，把人的注意力押在定义问题、设计验收和解释失败上。

软件工程暂时还没有死

为什么要招你，而不是买一个 $200 的 Claude Max?

我们有一个前所未有的机遇：一个人能像以前一个团队一样解决现实世界中的问题——保研的时候，希望大家能答上这个问题，除了你可以坐牢?

- ✦ 不要和 Agent 比产能。你的稀缺性是：方向 × 验证 × 责任——让一支可复制的 Agent 团队朝真实结果收敛。
- ✦ “可以坐牢”只点出了责任；责任必须和最终决策权、情境判断与停止权绑定。没有这些权力却只负责背锅的人，只是自动化系统的 moral crumple zone。
- ✦ 30 秒回答：我能找到值得解决的问题，把未知拆成便宜的实验，用证据让坏方向及时停下，并对长期结果负责。